RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

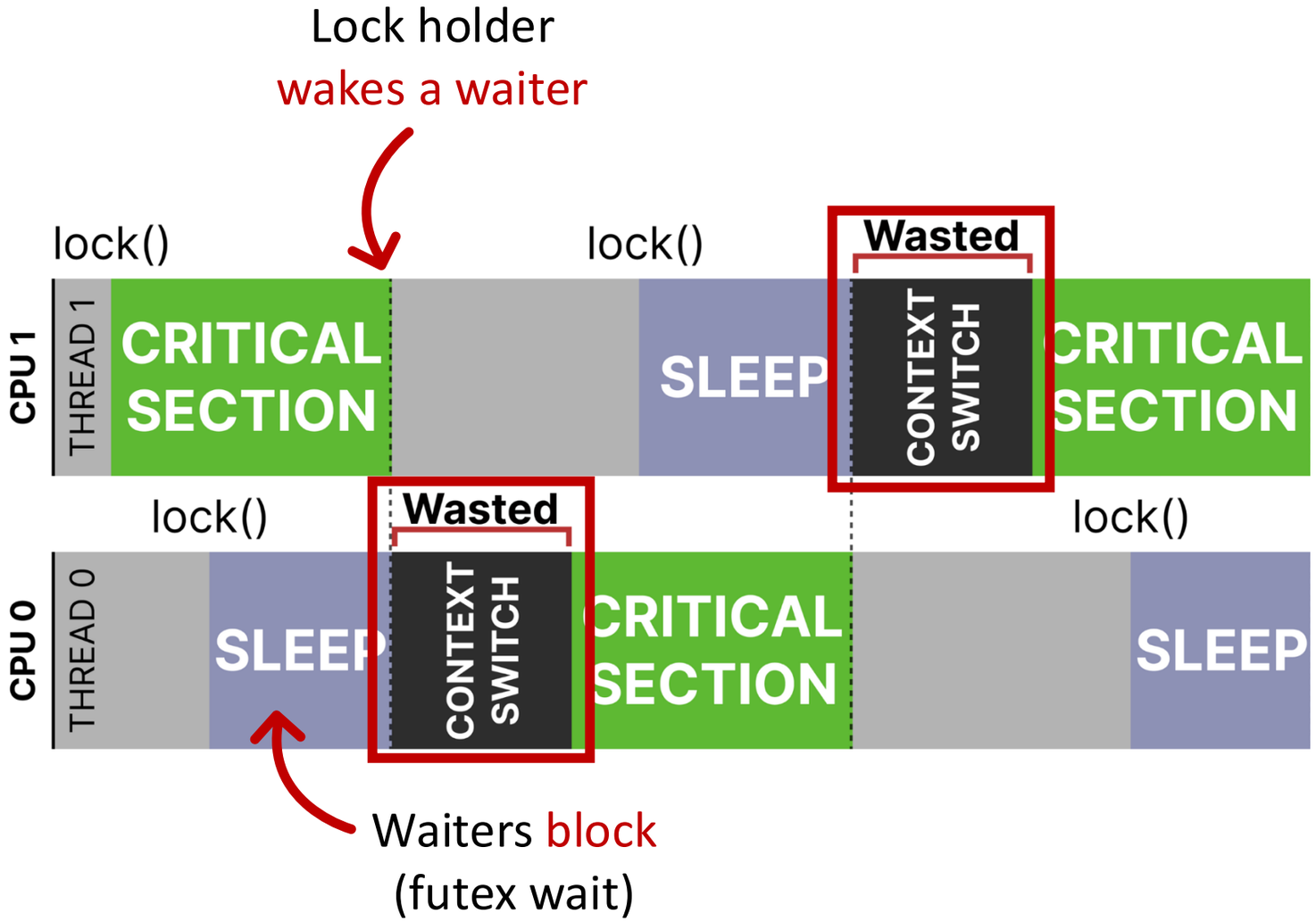
Inria

KERNEL SUPPORT FOR USER-SPACE LOCKING
HANDLING LOCK-HOLDER PREEMPTION

KERNEL RECIPES 2026, PARIS

Victor Laforet
Inria, Paris, France

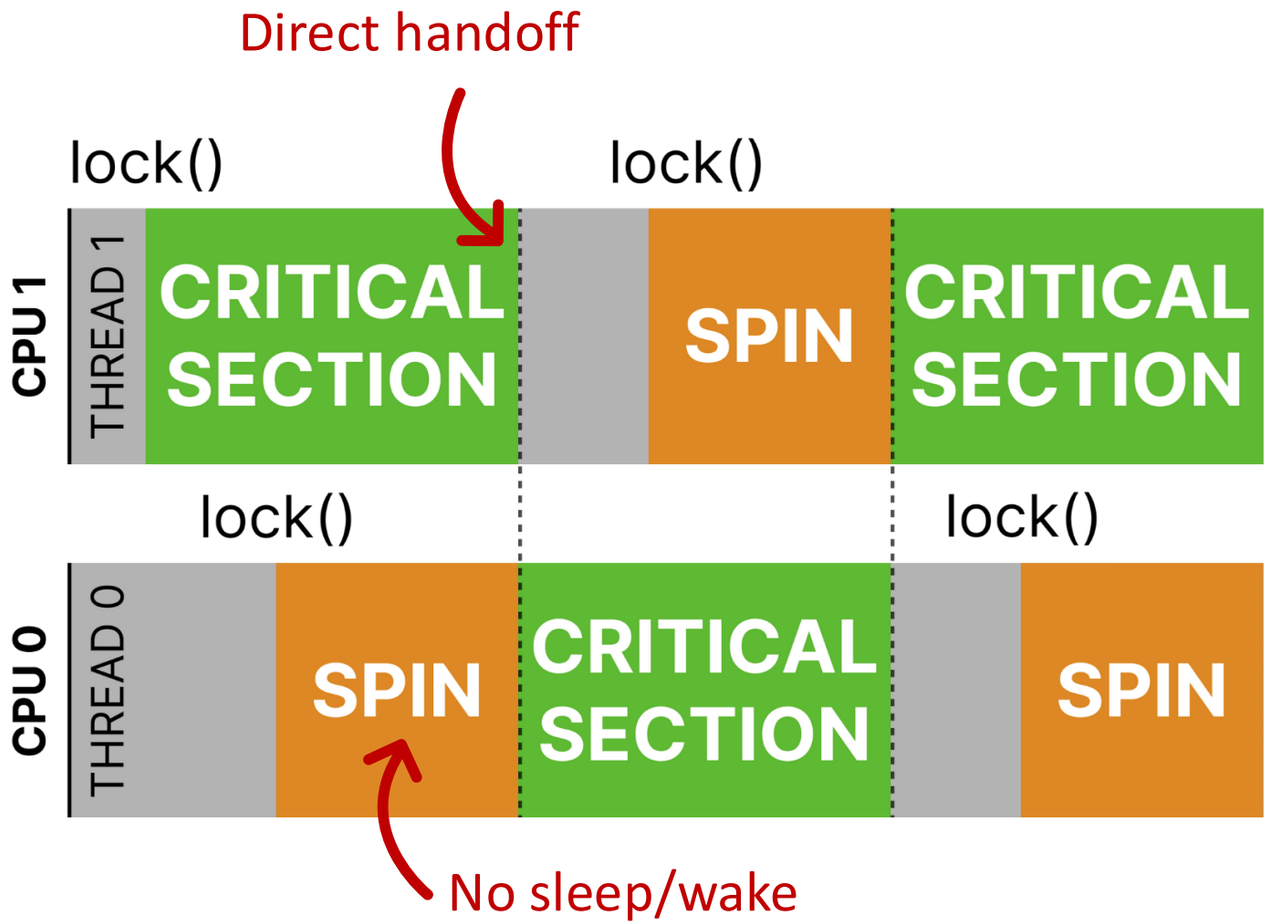
Blocking locks handoffs require sleeping and waking





Problem:
Sleeping and waking add latency to contended handoffs

Spinlocks avoid scheduler involvement



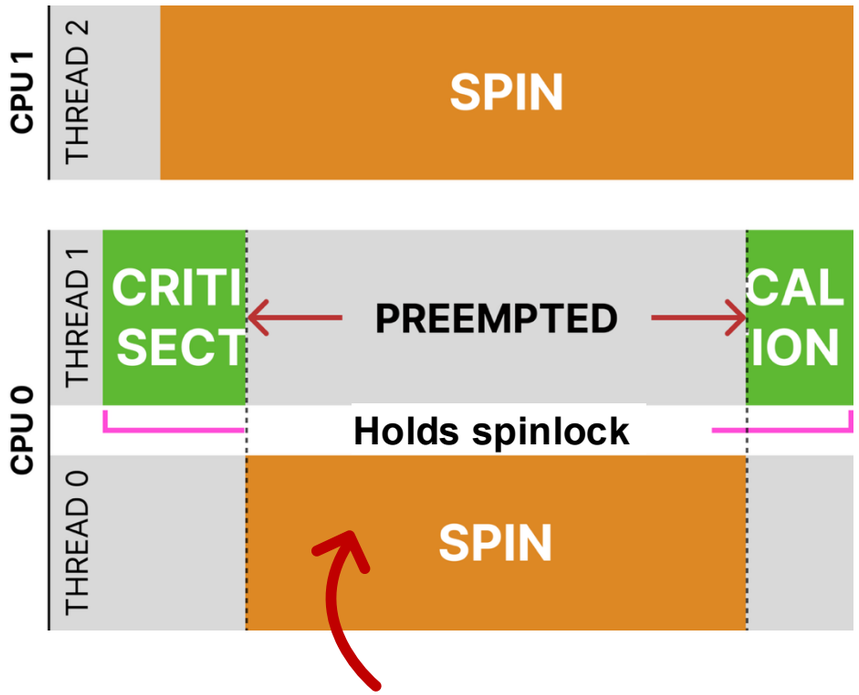
Simple TAS spinlock

```
void lock(state) {
  while (atomic_exchange_acquire(&state, LOCKED)
          != UNLOCKED)
    ;
}
```

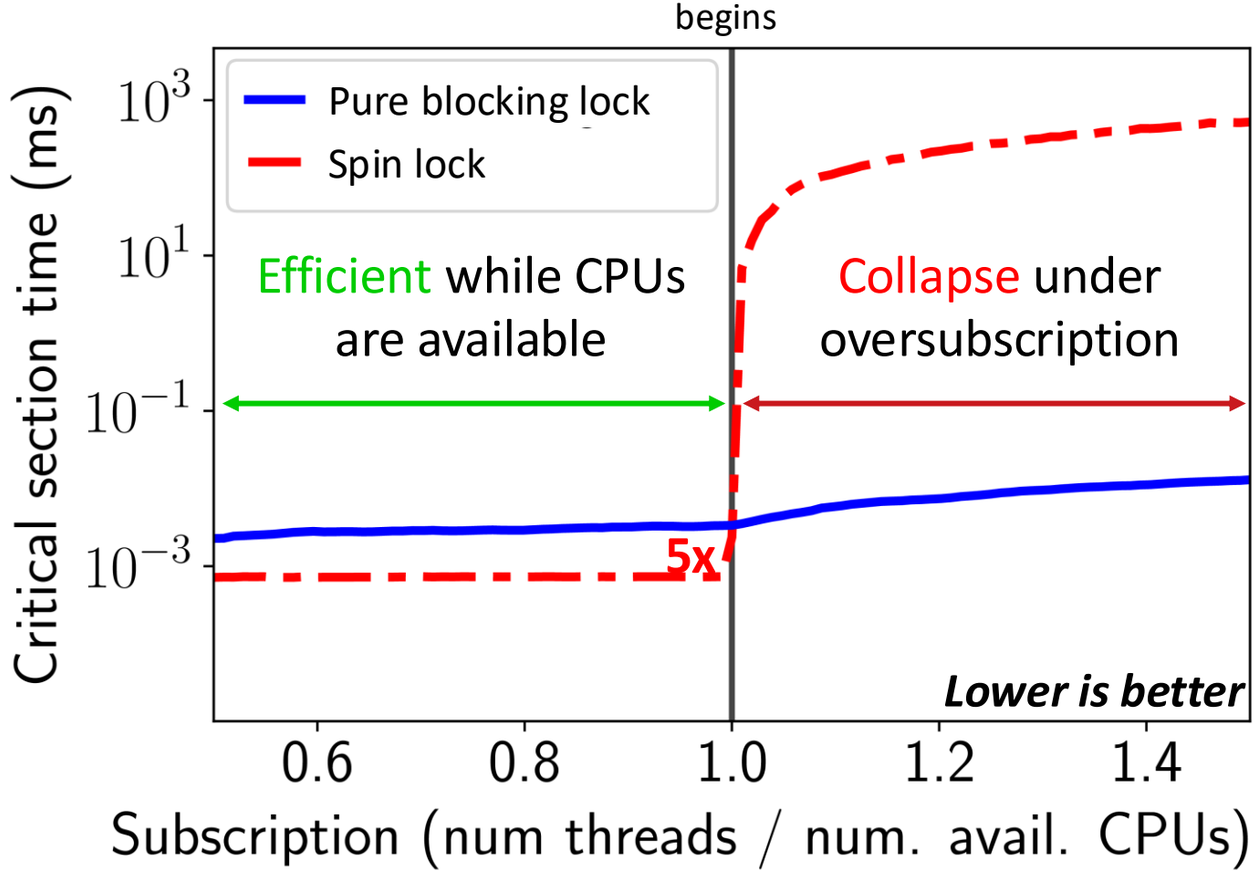
- No scheduler involvement on handoff
- Fast transitions between CS

… as long as the lock holder keeps running.

Lock-holder preemption causes performance collapse

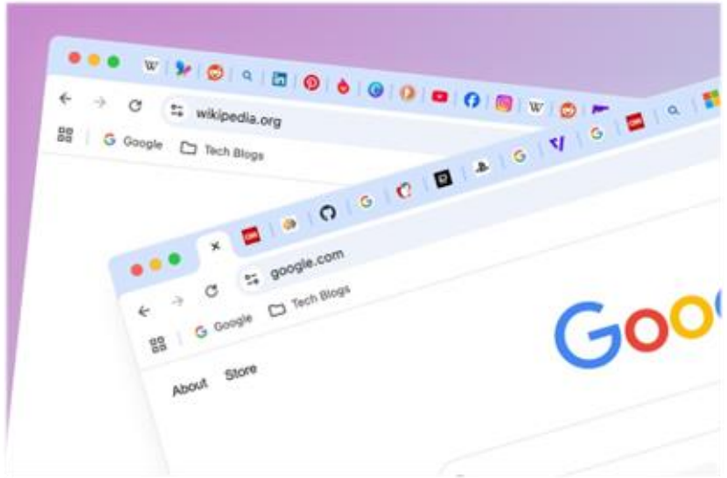


Waiter runs instead of lock holder



When the lock holder is preempted, waiters waste CPU and progress stalls.
=> Leads to a performance collapse under oversubscription (threads > cores)

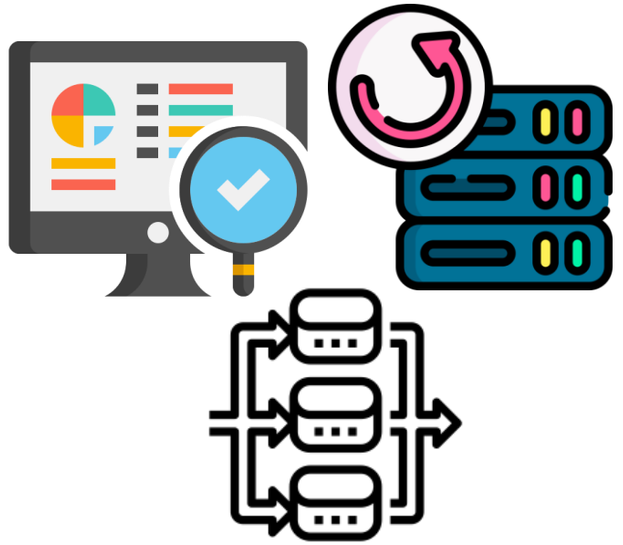
Applications cannot assume dedicated CPUs



Desktop applications: many concurrent processes



Production workloads: Meta's DCPerf: threads > cores



System activity: monitoring, backups, background services, …

User-space locks must remain robust under oversubscription

Time Slice Extension (TSE) keeps the critical section running



Idea: Prevent the scheduler from preempting the lock holder

1. Request TSE
2. Cancel TSE request
3. Yield if the extension was granted

Cheap: only writes, no sync

```
1  rseq->slice_ctrl.request = 1;

   // Prevent compiler reordering
   barrier();
   critical_section();
   barrier();

2  rseq->slice_ctrl.request = 0;
   if (rseq->slice_ctrl.granted)
3      rseq_slice_yield();
```

Using TSE to build a lock

1. Wait until the lock looks free
2. Request TSE before trying to acquire the lock
   No preemption window
3. On acquisition failure: cancel TSE request
   Yield if TSE was already granted



Request TSE at all times while holding the lock.

```
void lock(L) {
  while (true) {
1   while (atomic_load(&L) != UNLOCKED);

2   rseq->slice_ctrl.request = 1;
    barrier();


    if (xchg(&L, LOCKED) == UNLOCKED)
      return; // acquired, TSE active

    barrier();
3   rseq->slice_ctrl.request = 0;
    if (rseq->slice_ctrl.granted)
      rseq_slice_yield();
  }
}

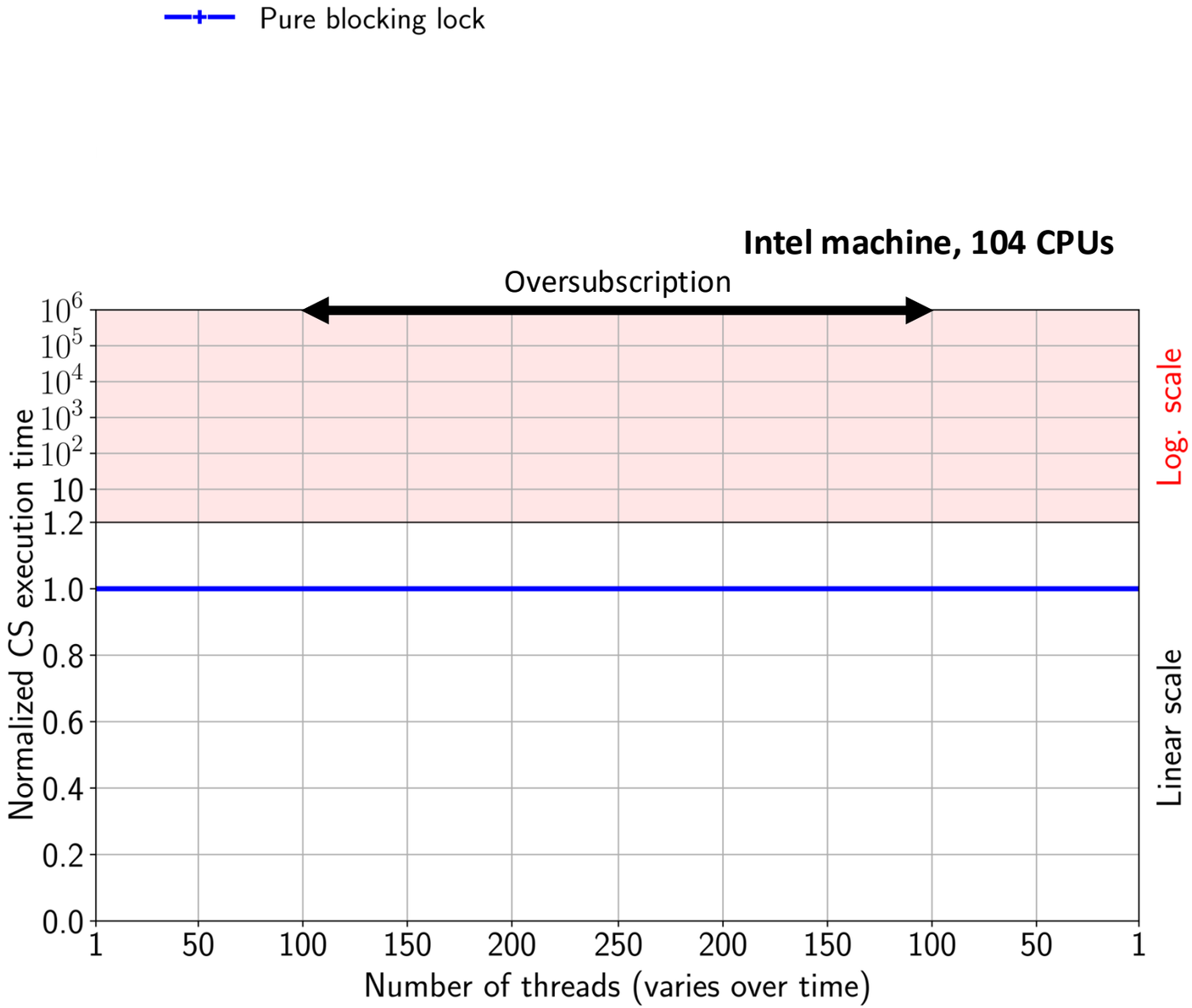
void unlock(L) {
  atomic_store(&L, UNLOCKED);

  barrier();
  rseq->slice_ctrl.request = 0;
  if (rseq->slice_ctrl.granted)
    rseq_slice_yield();
}
```

Single-lock microbenchmark

Measures latency function of number of threads

Pure blocking lock is the baseline

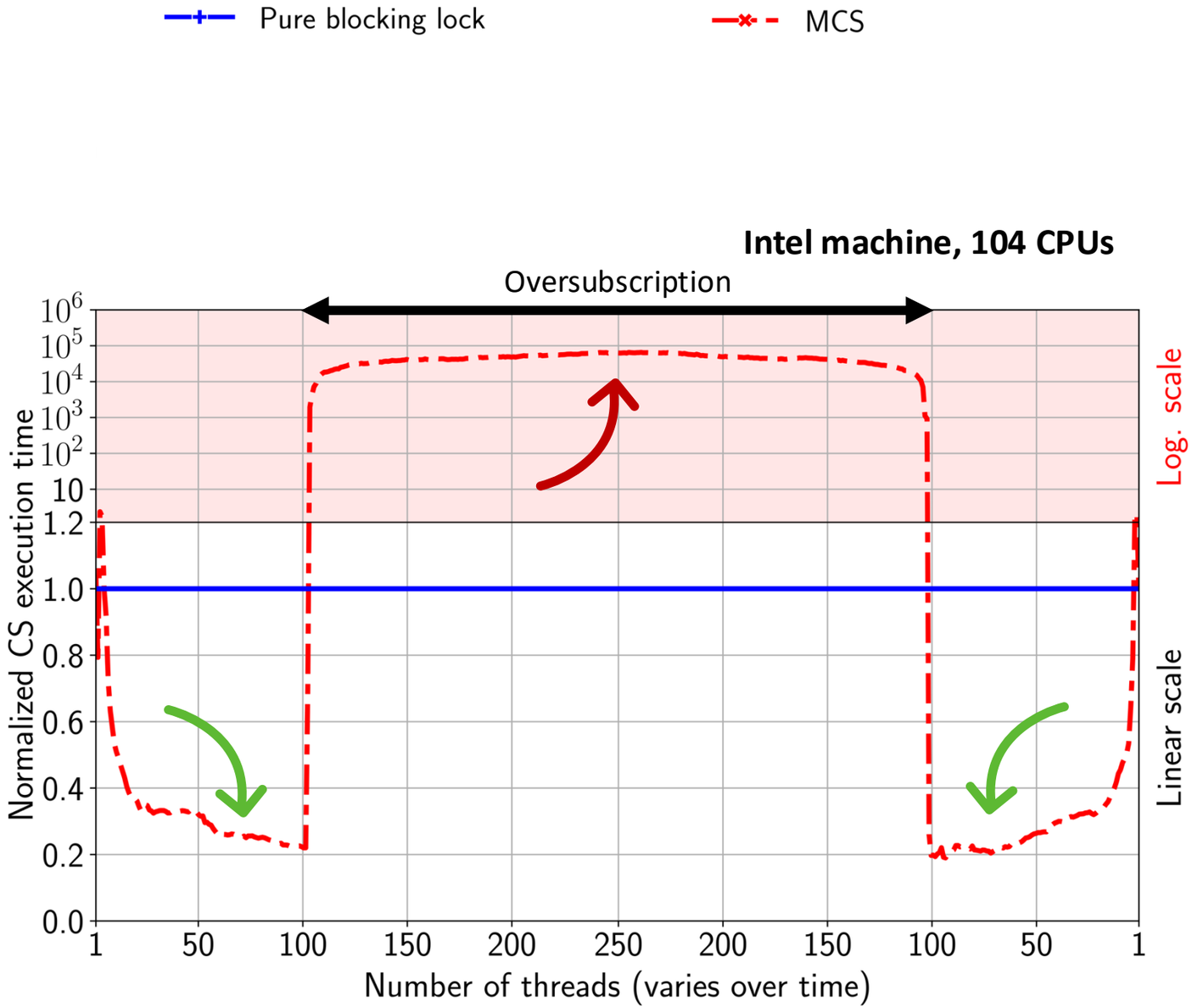


Lower is better

Single-lock microbenchmark

In non-oversubscription
MCS performs better than the pure blocking lock

In oversubscription
MCS collapses (unusable)

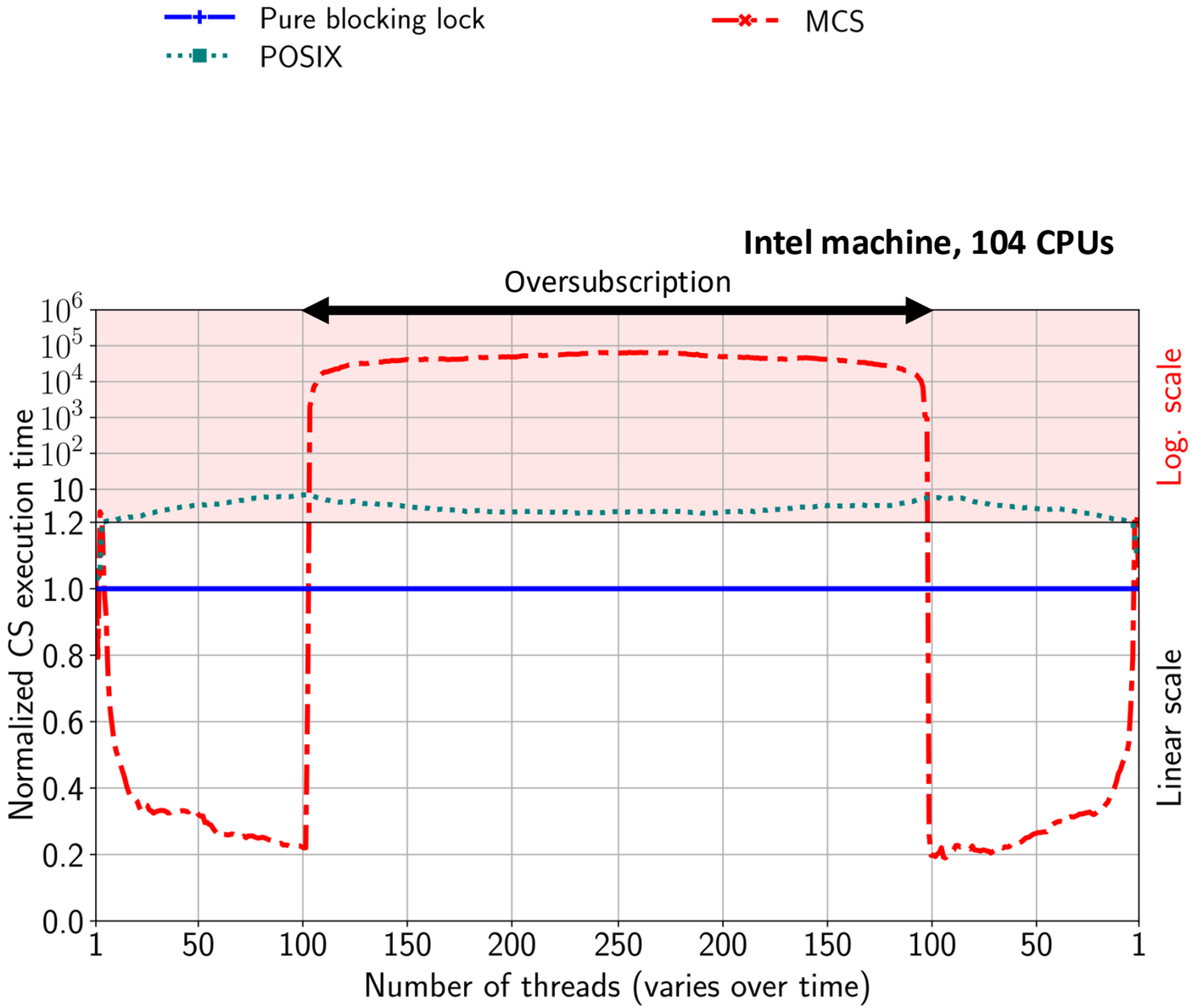


Lower is better

Single-lock microbenchmark

glibc's POSIX lock (*pthread_mutex_t*)

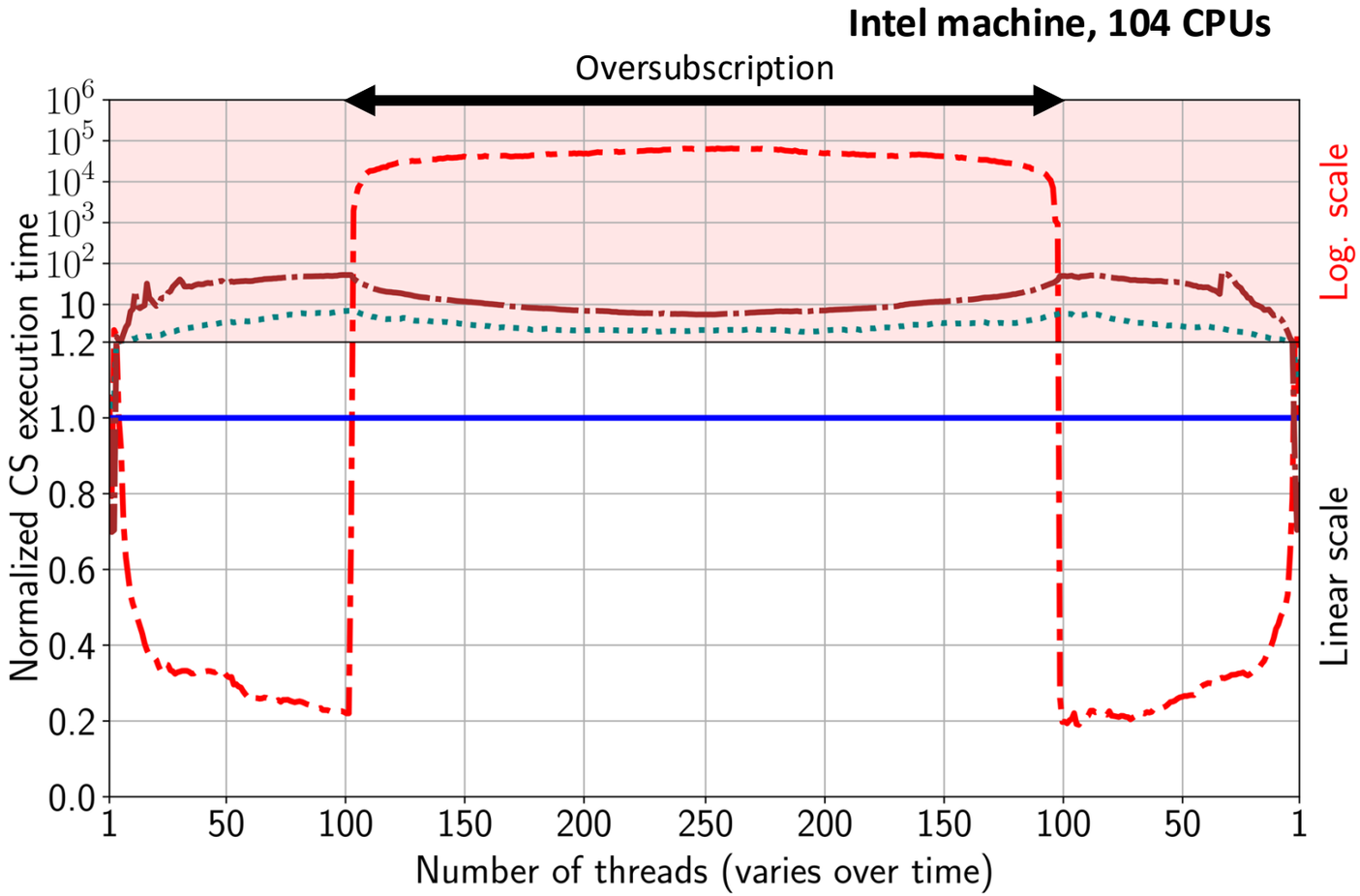
- Worse than pure blocking in that case



Lower is better

Single-lock microbenchmark

- TAS + TSE performs poorly
- TAS locks suffer from cache-line contention

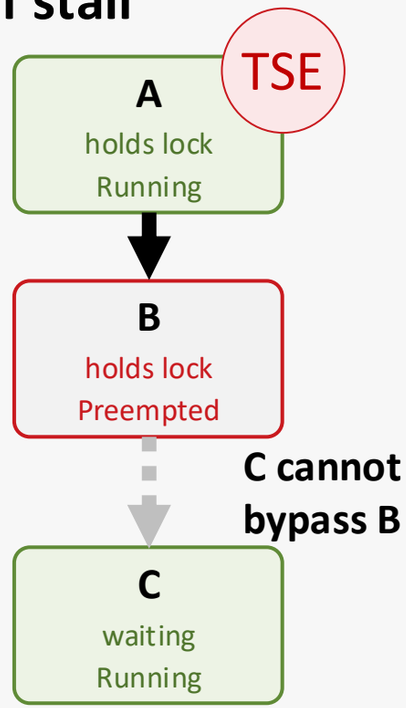


Lower is better

TSE cannot prevent next-waiter preemption

Ordered queue locks: handoff stall

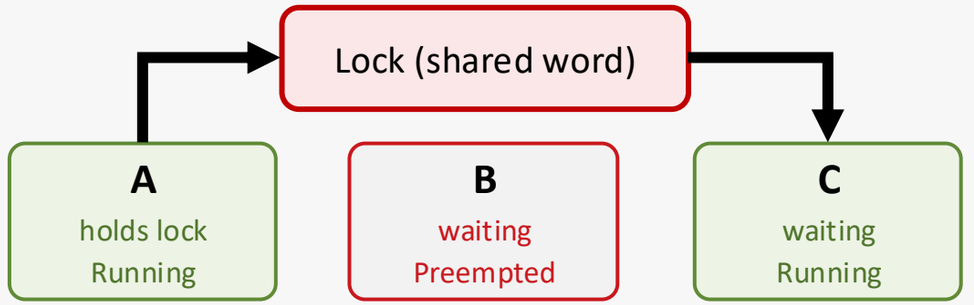
1. TSE keeps the holder A running
2. Unlock hands lock off to next waiter B
3. B becomes holder and progress stalls



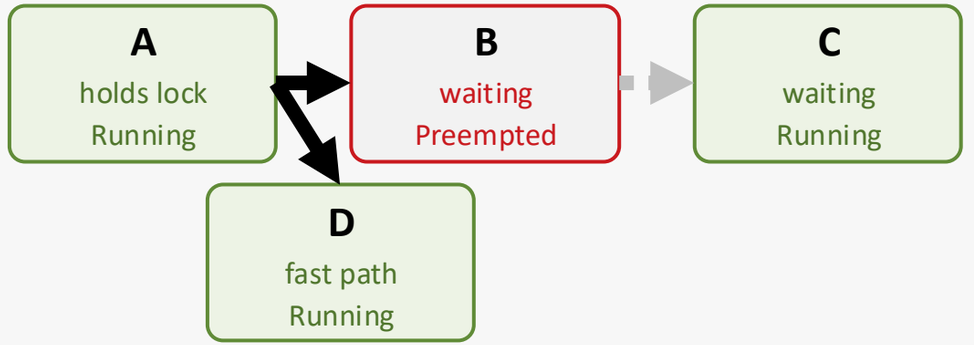
Our choice: modified qspinlock
queued slow-path + stealing fast path

Lock design implications for TSE

Option 1: Unordered acquisition (e.g. TAS/TATAS)



Option 2: Stealable fast path

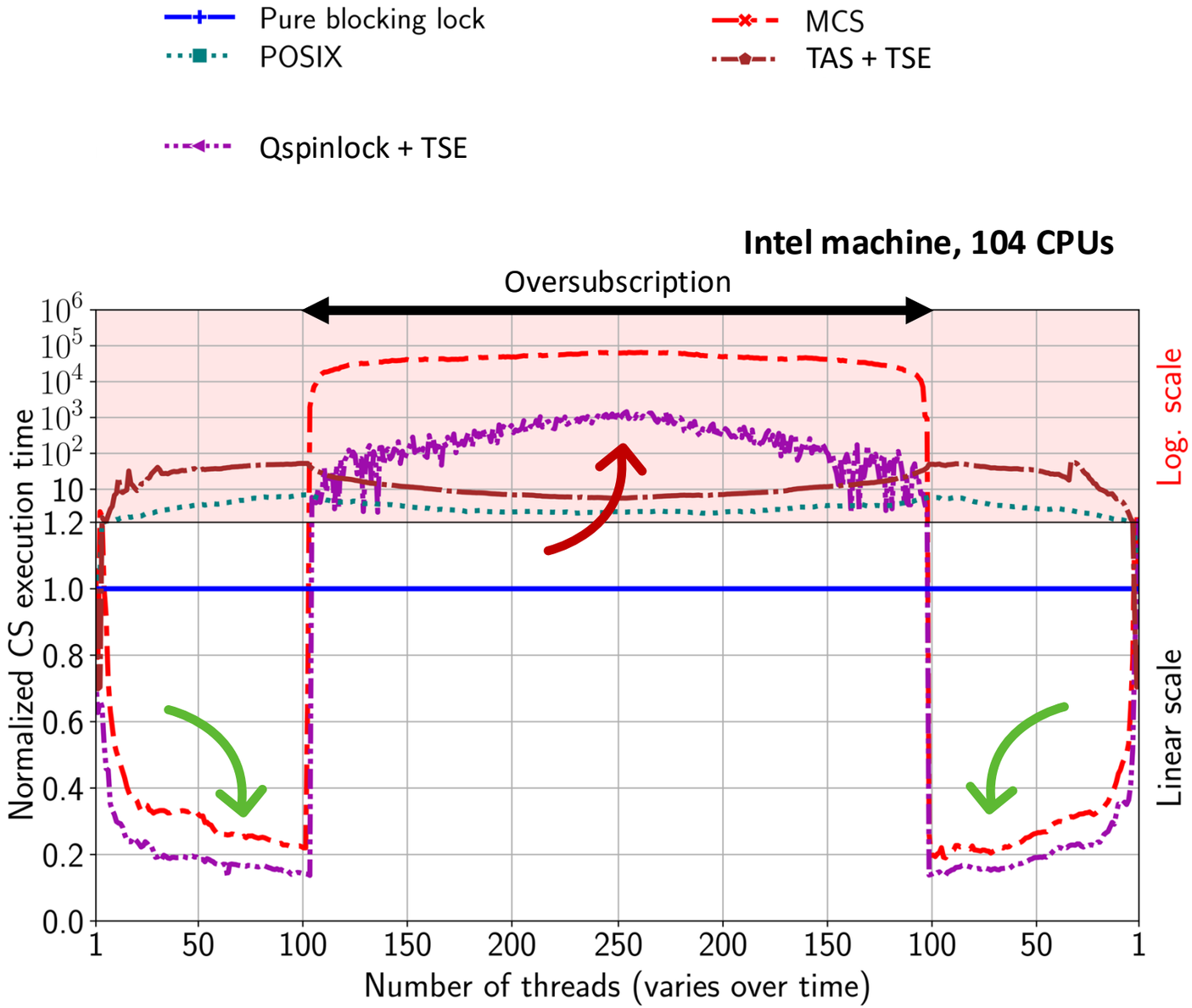


Single-lock microbenchmark

In non-oversubscription
Qspinlock + TSE performs similarly to MCS and better than blocking locks

In oversubscription
Collapse is reduced but still present

Protecting the holder is not enough to guarantee progress.

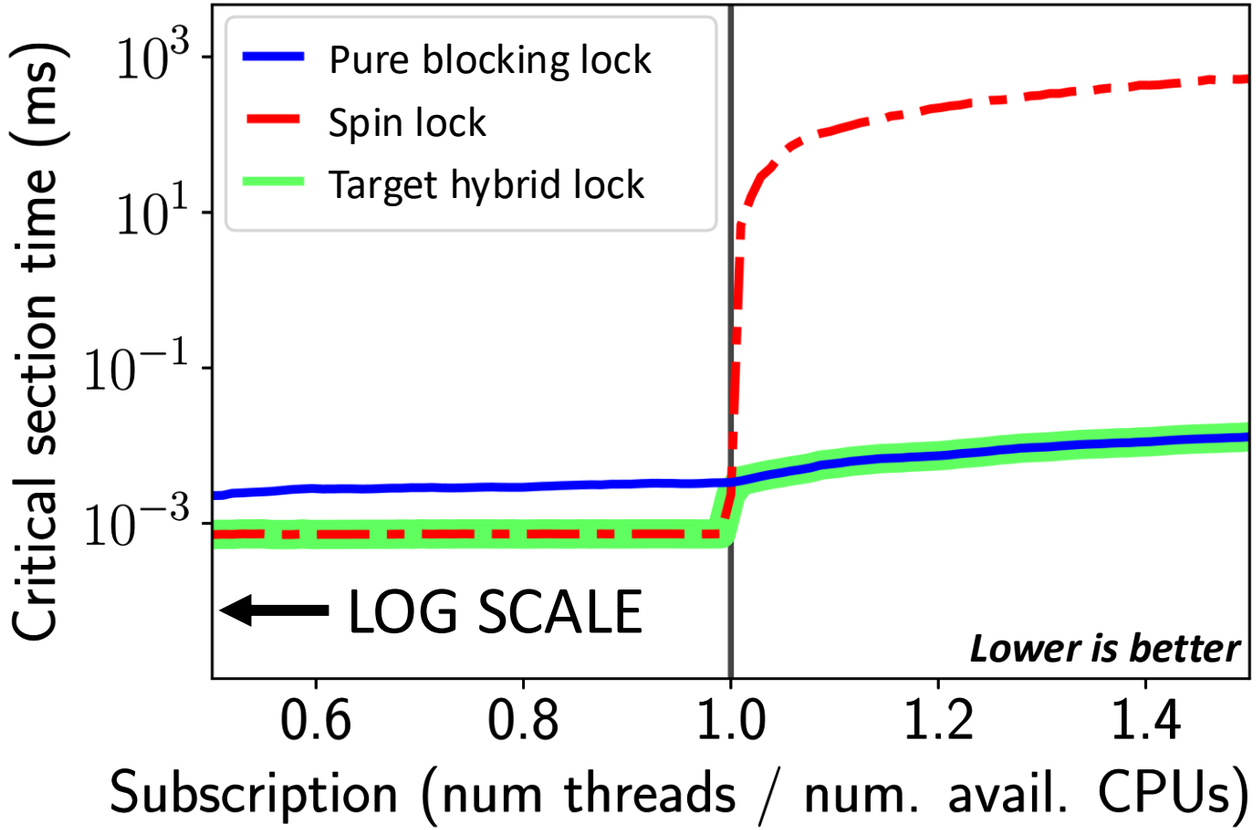


FlexGuard: Switching to blocking

Idea: Detect the CS preemption, and switch to a blocking lock

Can we do this?

Yes, with eBPF, we can accurately detect critical-section preemptions



FlexGuard: Preemption Monitor

- Detects critical-section (CS) preemptions
  - Provides a global counter of preempted CSs (`num_preempted_cs`)
  - eBPF handler hooks to the `sched_switch` event

- **How to detect that a thread is holding a lock?**
  - **Idea:** use a per-thread flag!
    - If flag set, we're in a critical section!
      (needs to be a counter for lock nesting)
  - **Is that enough to be accurate?**

Example: TATAS lock

```
5  def lock(L):
6    while (True):
7      if L == UNLOCKED:
9        if XCHG(&L, LOCKED) == UNLOCKED:
11         break
                                  <-
CS
13 def unlock(L):
                                  <-
16   L = UNLOCKED
```

FlexGuard: Preemption Monitor

Is it important to be fully accurate?

- Yes, a CS is often only a few instructions long
- Preemptions are likely in `lock()`/`unlock()`

⇒ Sufficient to cause performance collapse!

Example: TATAS lock

```
5  def lock(L):
6    while (True):
7      if L == UNLOCKED:
9        if XCHG(&L, LOCKED) == UNLOCKED:
11         break
12     cs_counter += 1
13 def unlock(L):
14     cs_counter -= 1
16     L = UNLOCKED
```

CS? → line 9

CS: lines 11–12

CS: lines 13–14

CS: line 16

Using FlexGuard to build a lock

- Based on Qspinlock (MCS queue + fast path using global lock word)

CS preemption detected



Spinning mode
⇔ num_preempted_cs == 0

- Waiters enter the MCS queue
- First spinner spins on the lock word instead of its node

Qspinlock

Blocking mode
⇔ num_preempted_cs > 0

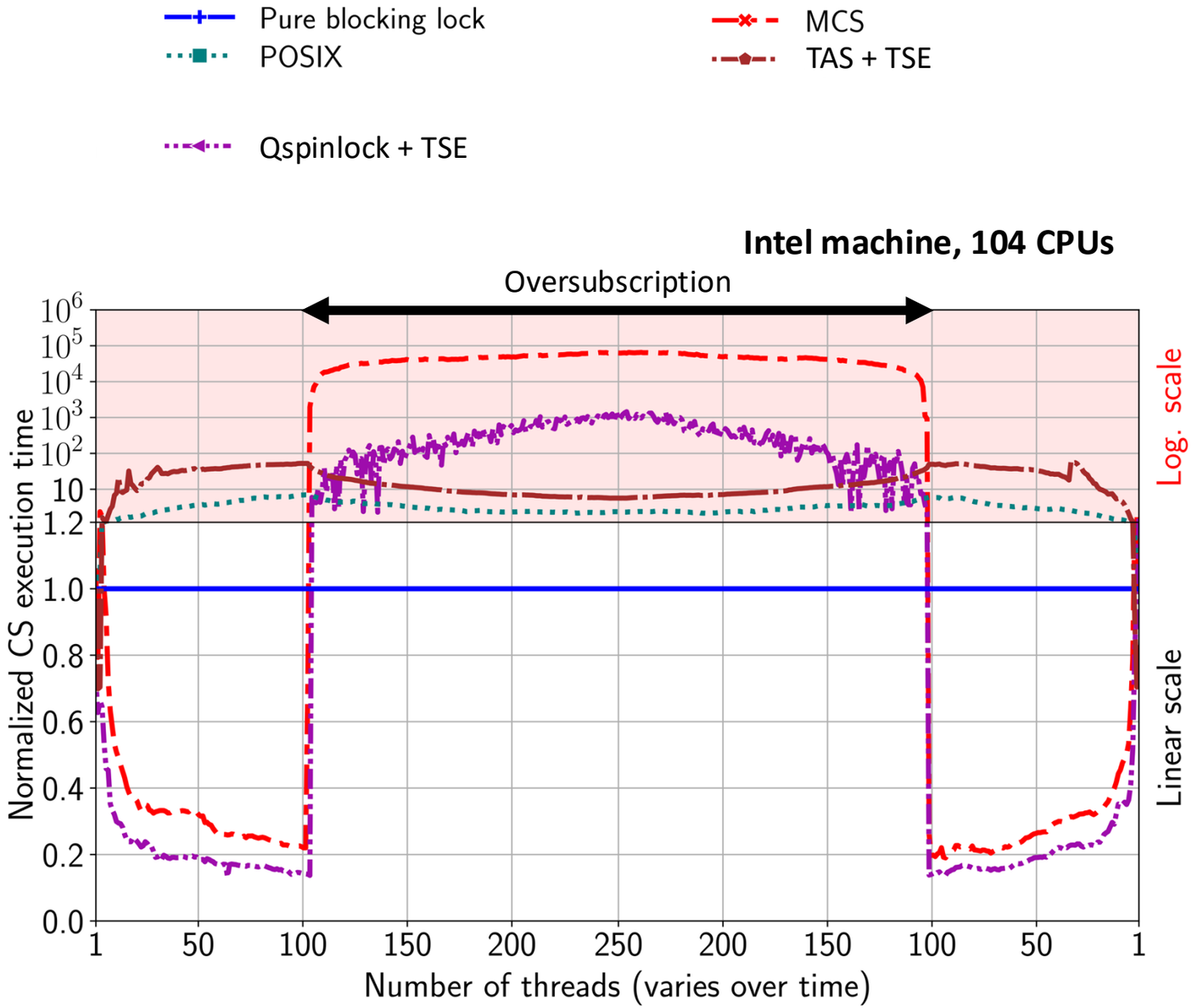
- Waiters bypass MCS queue
- Use global lock word as futex word

Futex blocking lock



No preempted CS remains

Single-lock microbenchmark

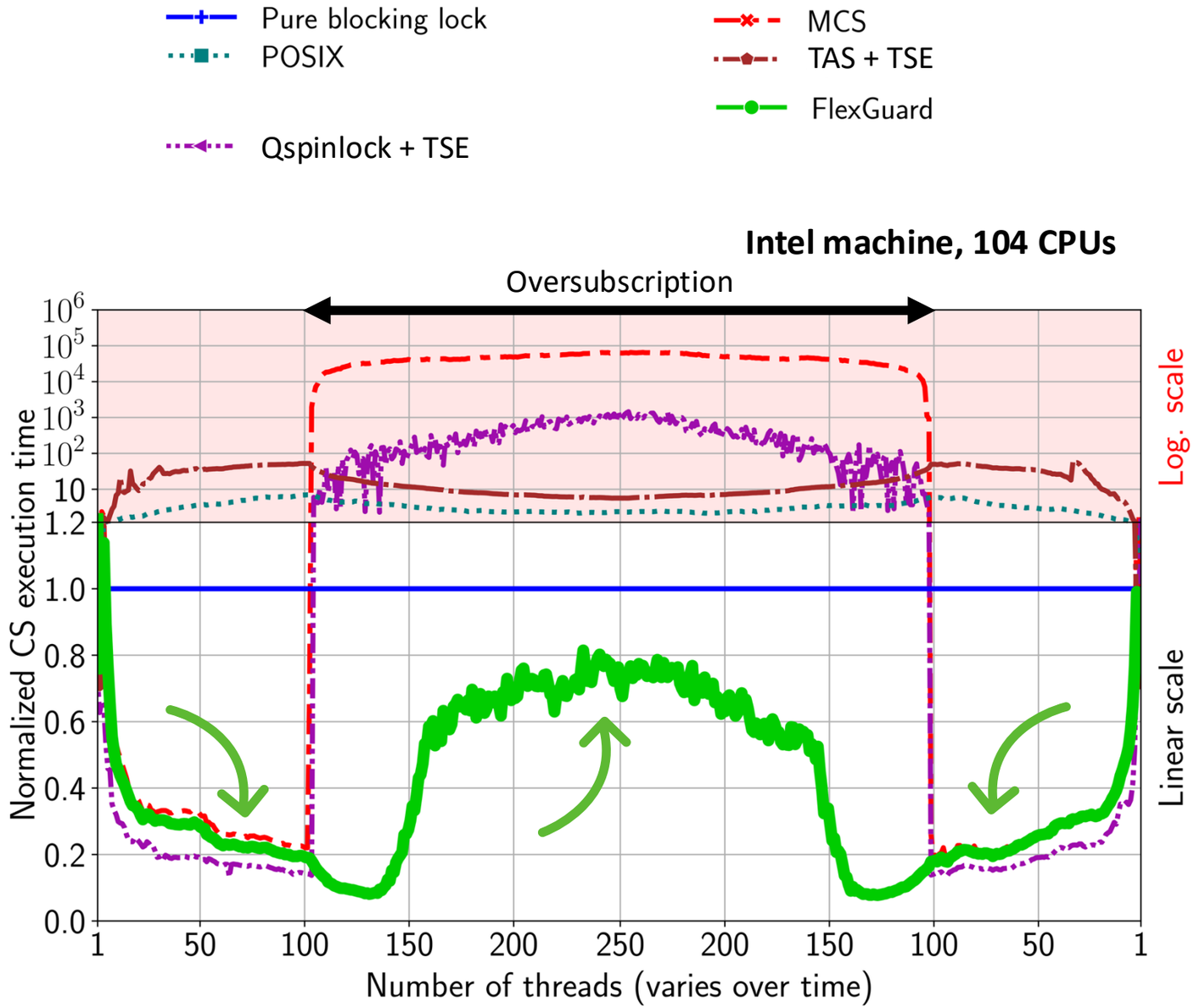


Lower is better

Single-lock microbenchmark

In non-oversubscription
FlexGuard performs similarly to MCS and Qspinlock + TSE

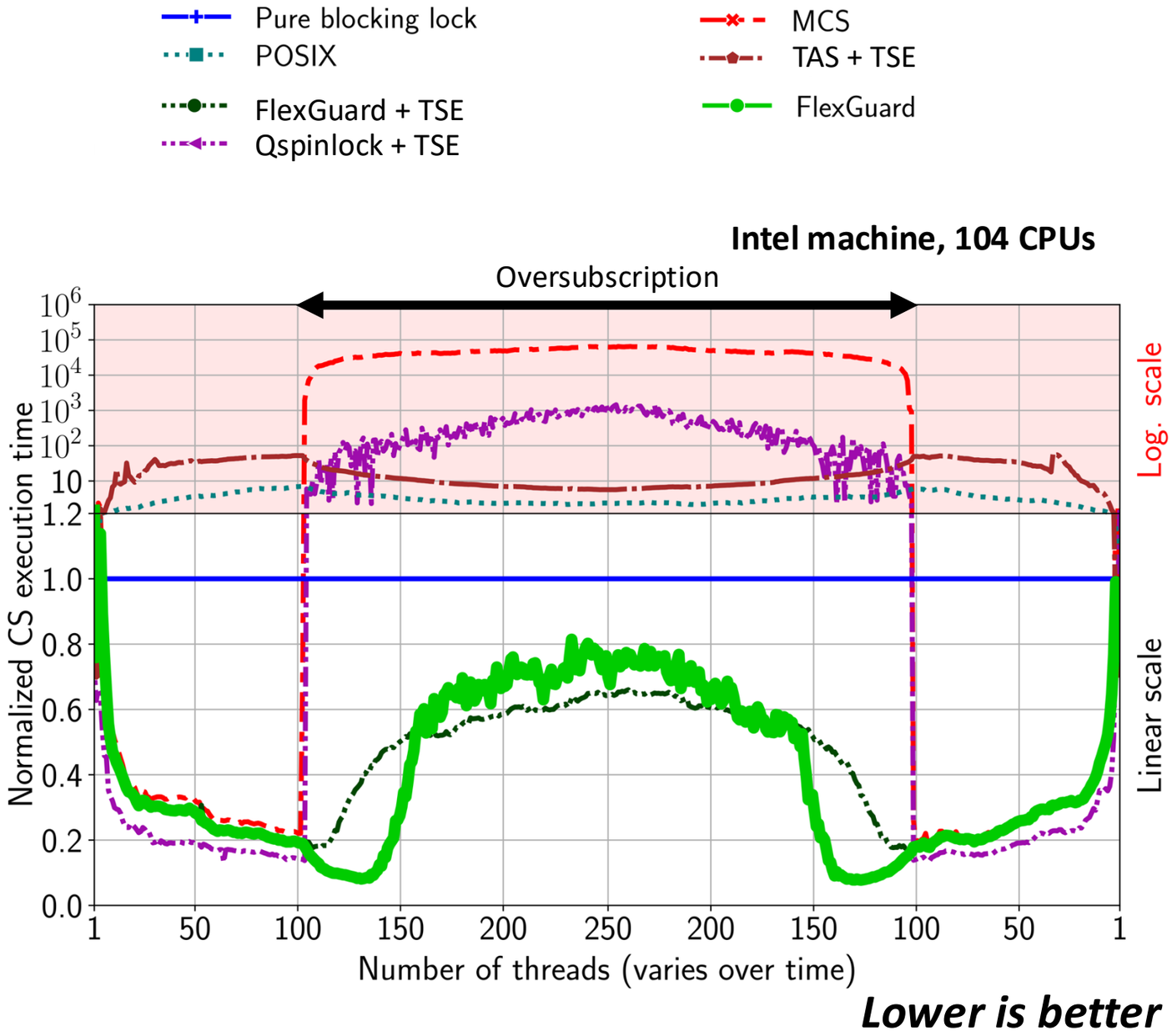
In oversubscription
FlexGuard avoids the performance collapse of other locks.



Single-lock microbenchmark

FlexGuard + Time Slice Extension

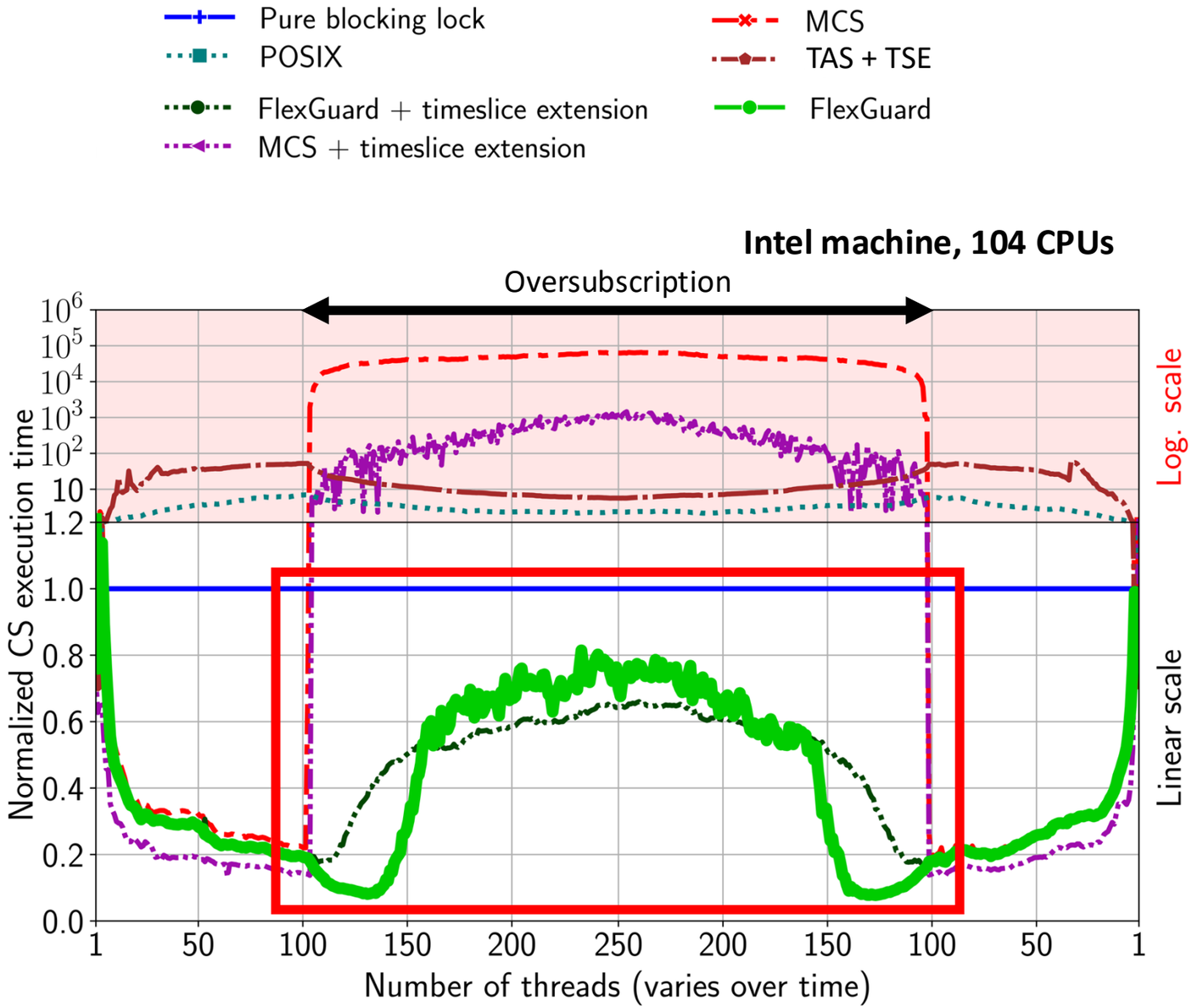
- performs similarly to FlexGuard
- more stable at high oversubscription



Lower is better

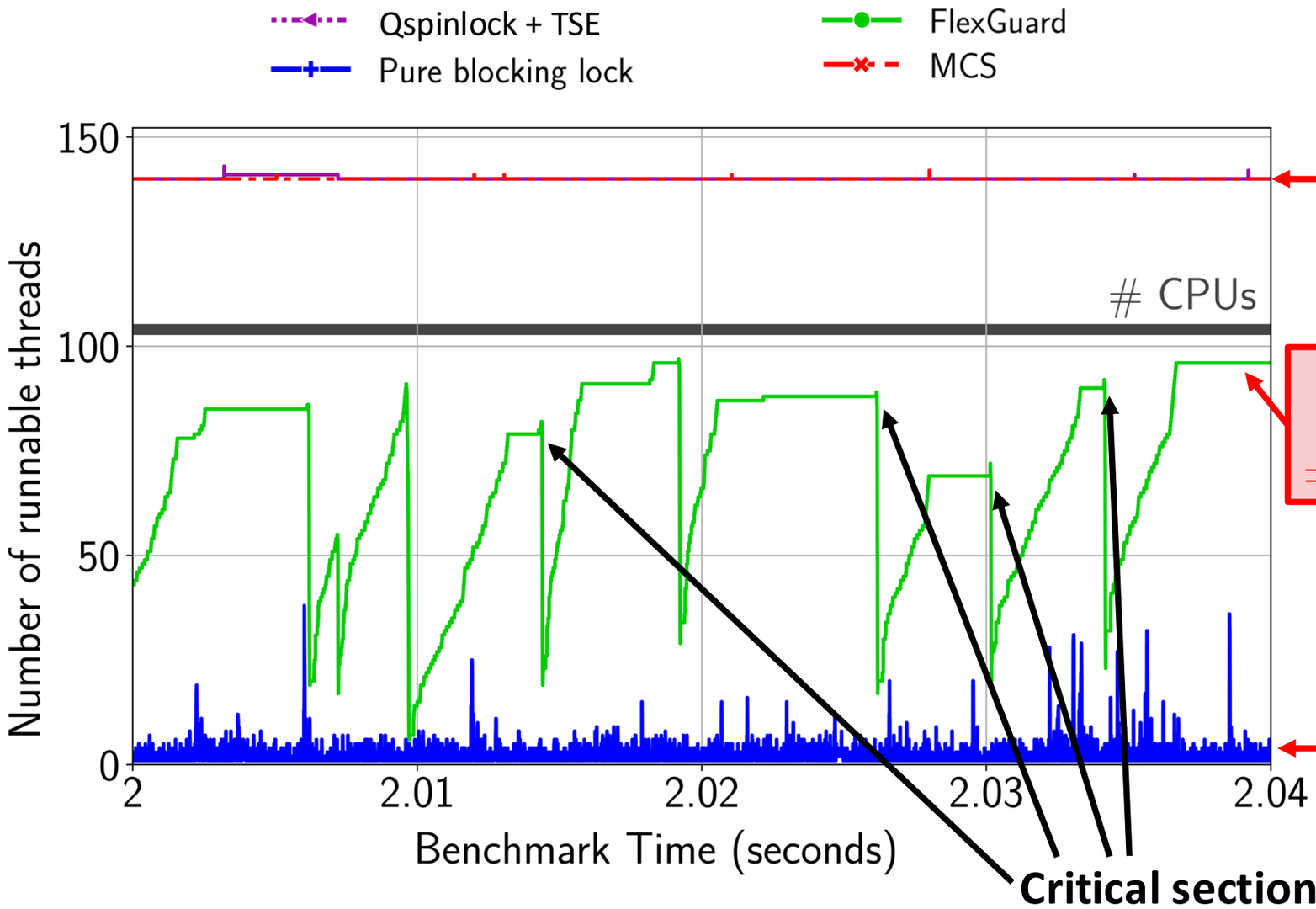
Single-lock microbenchmark

FlexGuard greatly outperforms the blocking locks… but why?



Oversubscribed lock behavior

Microbenchmark with 140 threads
Intel machine, 104 CPUs
⇒ Oversubscribed case

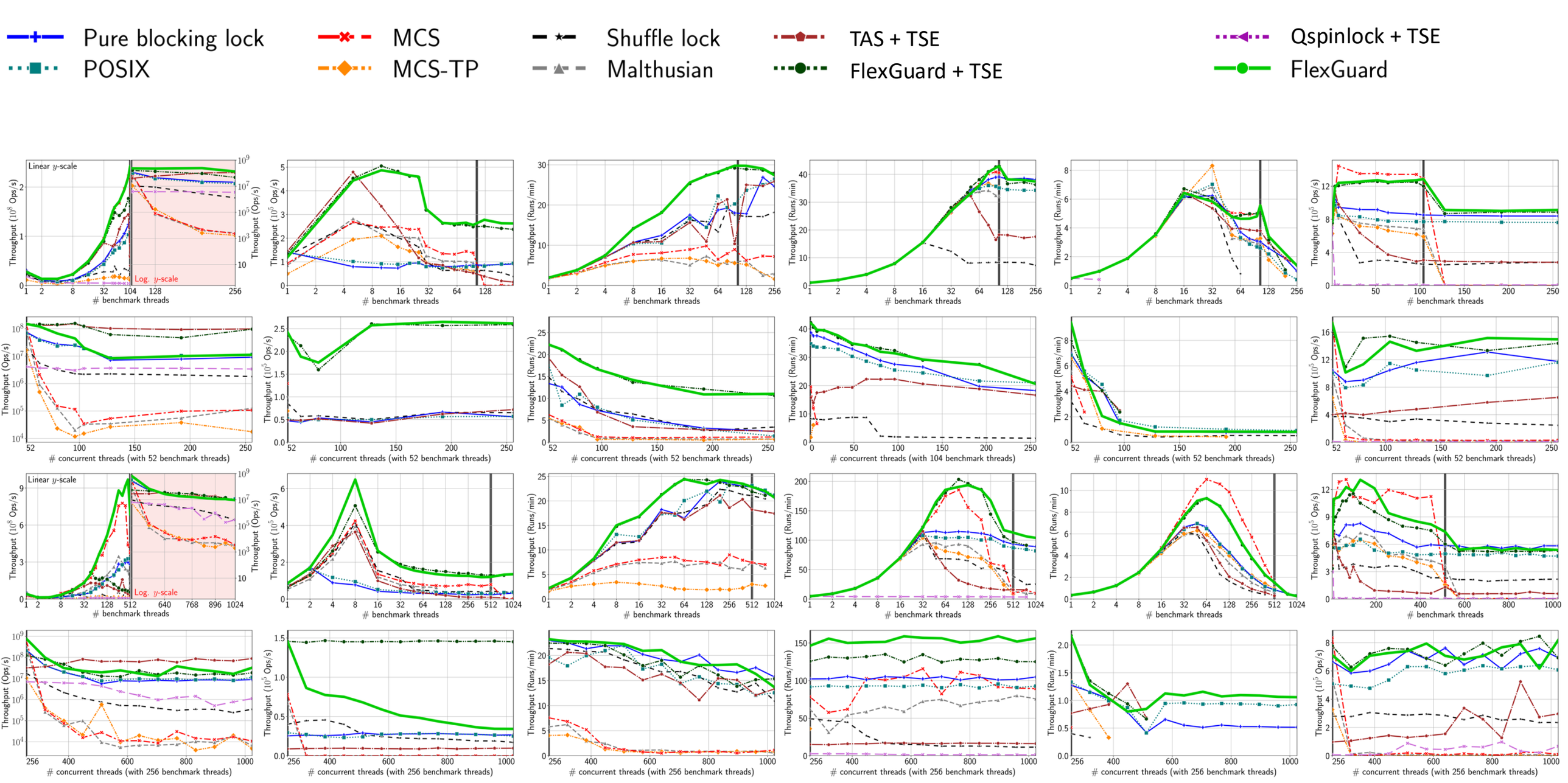


MCS performs poorly because
spinning waiters > # CPUs

FlexGuard performs best because
0 < # spinning waiters ≤ # CPUs
⇒ Uncollapsed spinlock behavior in oversubscription

Pure blocking lock performs better, but
the next waiter is not running, because
#spinning waiters = 0

Critical section preemptions (Spinning ⇒ Blocking)



Higher is better

Evaluation



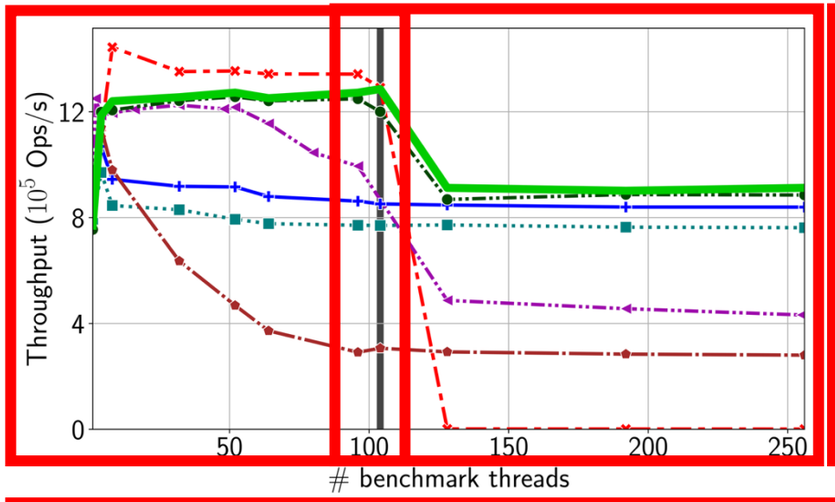
In the oversubscribed case,

- Qspinlock + TSE does not avoid the performance collapse
- FlexGuard outperforms blocking locks

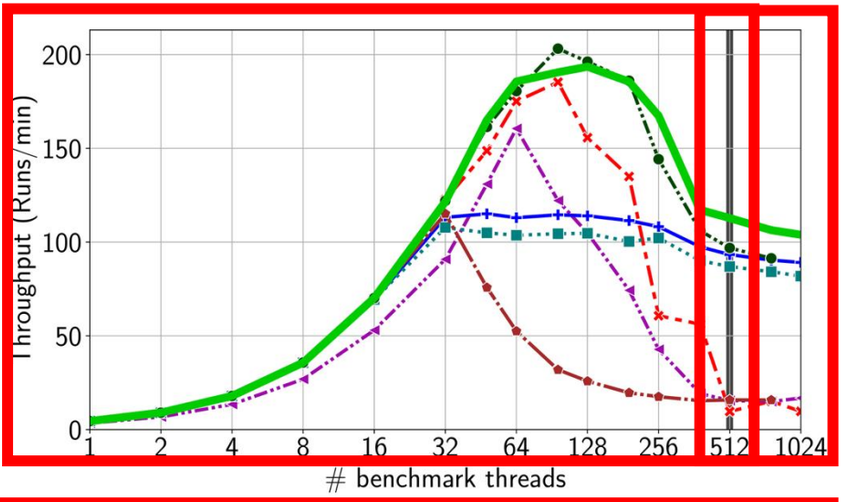
Benchmarks w/ concurrent workload: fixed number of benchmark threads, varying number of concurrent threads

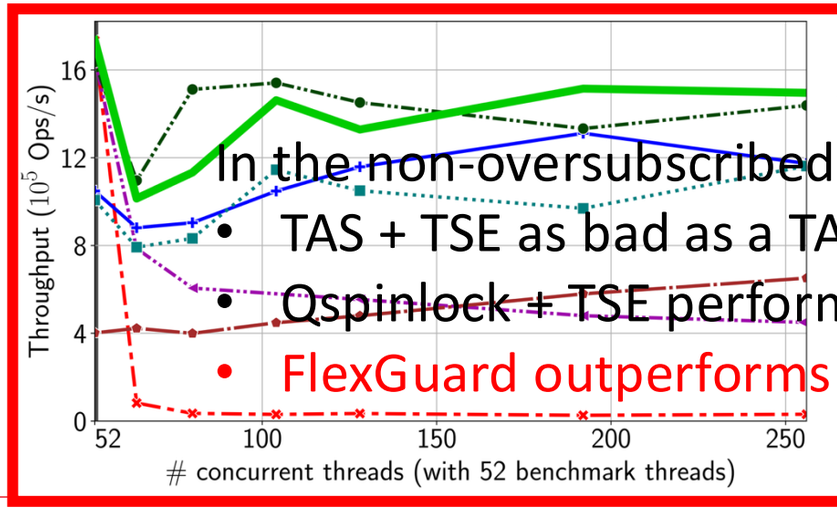
⇒ **Oversubscribed case**

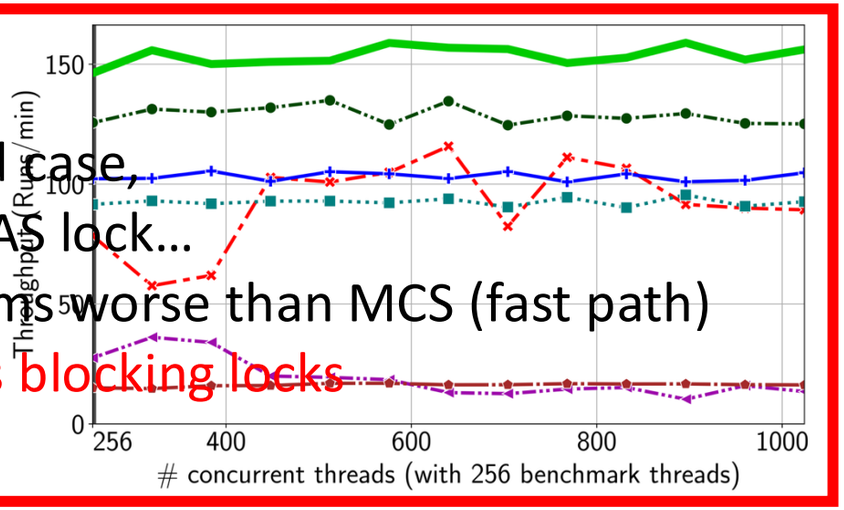
LevelDB *readrandom*
Intel (104 hw threads)



Raytrace
AMD (512 hw threads)







In the non-oversubscribed case,

- TAS + TSE as bad as a TAS lock…
- Qspinlock + TSE performs worse than MCS (fast path)
- FlexGuard outperforms blocking locks

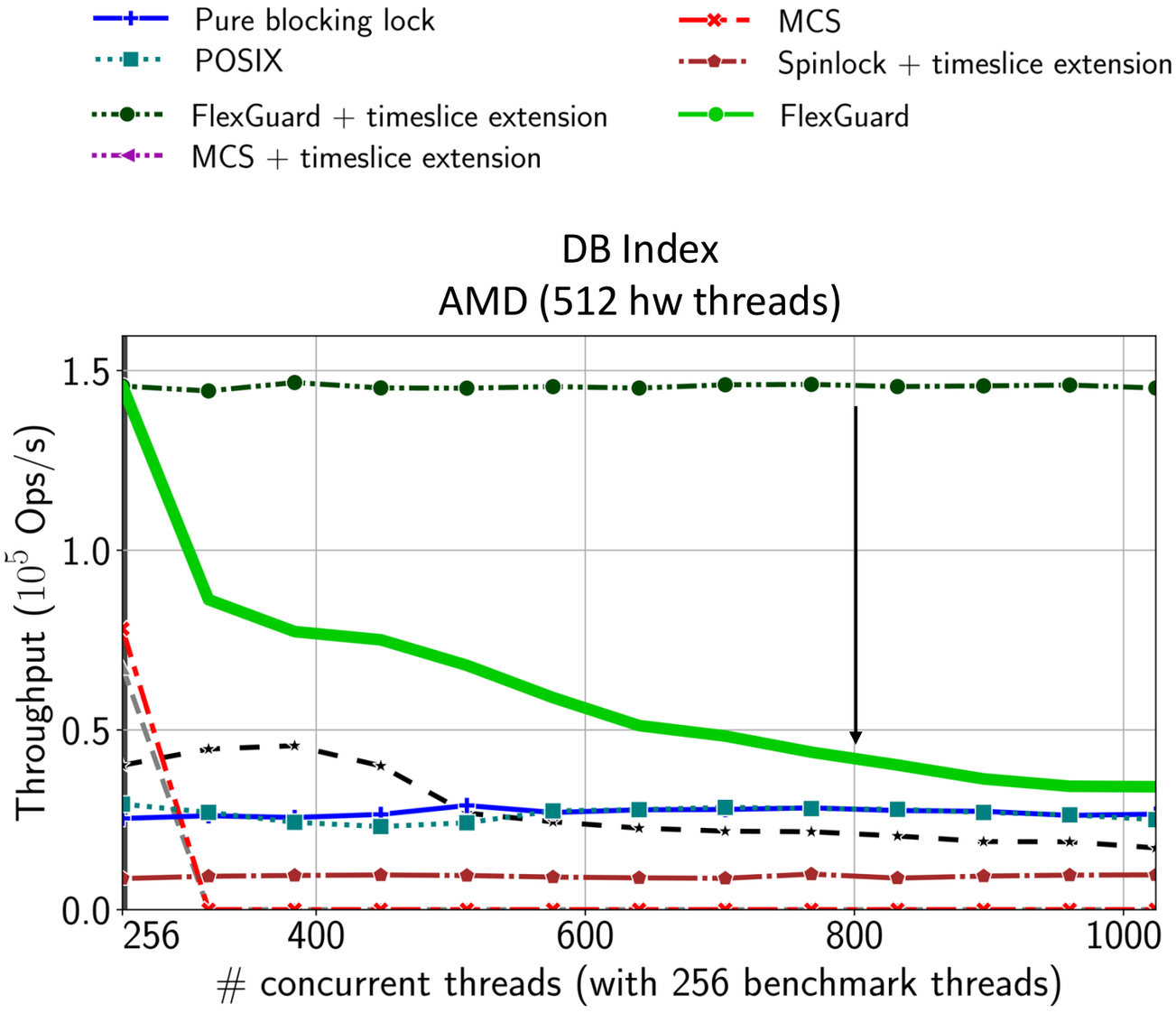
Higher is better

FlexGuard + Time Slice Extension

FlexGuard is limited by its ability to put threads to sleep

When the number of waiters is low, lock holder may not get re-scheduled quickly enough

⇒ FlexGuard + Time Slice Extension more consistently ensures good performance



Conclusion

Time Slice Extension

- Lets user-space request extra time before preemption
- Reduces critical section preemptions
- 10% faster than blocking locks when oversubscribed under very low contention
- Requires specific lock designs

FlexGuard

- Enables user-space spinlocks in oversubscription
- Delivers spinlock-like behavior without collapse
- Combines the reliability of blocking with the performance of spinning

FlexGuard + Time Slice Extension

Best solution in our scenarios, even when the number of waiters is limited

Thank you!